MP4: Page Manager II and Virtual Memory Pool

Mohammed Sharique
UIN: 837004870
CSCE611: Operating System

03/18/2026

Assigned Tasks

Main: Completed.

System Design

The goal of this machine problem was to extend the paging system to support large address spaces and implement a virtual memory allocator. The key idea was to move page table pages from directly mapped memory into mapped memory and use recursive page table lookup to manage them efficiently. Along with this, I implemented a lazy virtual memory allocator that allocates regions in virtual memory but delays physical frame allocation until a page fault occurs.

- I modified the page table structure to support recursive mapping by making the last entry of the page directory point to the page directory itself. This allows accessing both page directory entries and page table entries using fixed logical addresses without needing separate mappings.
- Using this recursive mapping, I implemented functions to compute the logical addresses of PDEs and PTEs directly. This removed the need to manually traverse page tables or rely on physical addresses.
- Page faults are now handled by first checking whether the faulting address is legitimate. This includes checking if the address belongs to any registered virtual memory pool. If valid, the page table allocates page tables and frames on demand using the process memory pool.
- I implemented a virtual memory pool (VMPool) which manages memory regions in virtual address space. Instead of allocating frames immediately, it only tracks allocated and free regions. Physical memory is allocated only when a page is accessed and a page fault occurs.
- The VMPool maintains two arrays: allocated regions and free regions. These are stored inside the pool's own virtual memory space. The first page of the pool is reserved to store this metadata.
- Allocation is done in page-sized units by splitting free regions, while deallocation returns regions back to the free list and informs the page table to free any associated physical frames.
- The page table maintains a list of registered memory pools so that during a page fault it can validate whether an address is legitimate by querying each pool.

Code Description

I modified page_table.C and implemented a new module vm_pool.C and vm_pool.H. I reused cont_frame_pool from the previous machine problem. The code compiles by running make in the project directory.
Github: https://github.com/sharique-tamu/csce-611-machine-problems

page_table.C:PDE_address : This function computes the logical address of the page directory entry corresponding to a given virtual address using recursive mapping. It uses the last page directory entry which maps the page directory into virtual memory and returns a pointer to the required PDE.

```
unsigned long *PageTable::PDE_address(unsigned long addr) {
  unsigned long X = addr >> 22;
  return (unsigned long *)(0xFFFFF000 + (X << 2));
}
```

page_table.C:PTE_address : This function computes the logical address of the page table entry for a given virtual address. It uses the recursive mapping region to directly access the appropriate page table and returns a pointer to the PTE.

```
unsigned long *PageTable::PTE_address(unsigned long addr) {
  unsigned long X = addr >> 22;
  unsigned long Y = (addr << 10) >> 22;
  return (unsigned long *)(0xFFC00000 + (X << 12) + (Y << 2));
}
```

page_table.C:handle_fault : This function handles page faults for dynamically allocated memory. It reads the faulting address from CR2 and checks if the address is legitimate by querying registered VM pools. If the corresponding page directory entry is not present, a new page table is allocated and initialized. Then, if the page table entry is not present, a frame is allocated from the process memory pool and mapped. The TLB is flushed after updating entries.

page_table.C:register_pool : This function registers a virtual memory pool with the page table. It stores pointers to pools so that legitimacy of addresses can be checked during page faults.

page_table.C:free_page : This function frees a mapped page by locating its PTE, releasing the corresponding physical frame back to the frame pool, and invalidating the entry.

vm_pool.C:VMPool (constructor) : This constructor initializes a virtual memory pool with a base address and size. It sets up arrays for allocated and free regions within the pool's own virtual memory space. The first page is reserved for metadata. The pool registers itself with the page table before accessing its memory to avoid faults during initialization.

vm_pool.C:allocate : This function allocates a region of memory by finding a suitable free region and splitting it. Allocation is done in page-sized units. It updates both allocated and free region arrays but does not allocate physical frames immediately.

vm_pool.C:release : This function releases an allocated region by moving it back to the free list. It also informs the page table to free any pages that were mapped for this region.

vm_pool.C:is_legitimate : This function checks whether a given address belongs to a valid allocated region. It also allows access to the metadata page to avoid bootstrapping issues during page fault handling.

```cpp
void PageTable::handle_fault(REGS *_r) {
  (void)_r;
  unsigned long fault_address = read_cr2();

  bool legitimate = (fault_address < shared_size);
  if (!legitimate) {
    for (unsigned int i = 0; i < current_page_table->n_registered_pools; i++) {
      if (current_page_table->registered_pools[i]->is_legitimate(
              fault_address)) {
        legitimate = true;
        break;
      }
    }
  }

  if (!legitimate) {
    Console::puts("SEGMENTATION FAULT: illegitimate memory access\n");
    abort();
  }

  unsigned long dir_idx = fault_address >> 22;
  unsigned long *pde = current_page_table->PDE_address(fault_address);

  if ((*pde & 1) == 0) {
    unsigned long new_frame = process_mem_pool->get_frames(1);
    assert(new_frame != 0);
    *pde = (new_frame * PAGE_SIZE) | 3;

    unsigned long *new_pt = (unsigned long *)(0xFFC00000 + (dir_idx << 12));
    for (int i = 0; i < 1024; i++) {
      // Enabling R/W bit and invalid bit
      new_pt[i] = 2;
    }
  }
```

```cpp
void PageTable::register_pool(VMPool *_vm_pool) {
  assert(_vm_pool != nullptr);
  assert(n_registered_pools < MAX_VM_POOLS);
  registered_pools[n_registered_pools] = _vm_pool;
  n_registered_pools++;
  Console::puts("registered VM pool\n");
}
```

```cpp
void PageTable::free_page(unsigned long _page_no) {
  unsigned long page_addr = _page_no * PAGE_SIZE;
  unsigned long *pde = current_page_table->PDE_address(page_addr);
  if ((*pde & 1) == 0) {
    Console::puts("Freeing incorrect page\n");
    return;
  }
```

Testing

For this machine problem I relied on the provided tests for validating the implementation. The test repeatedly allocates and frees memory regions using the virtual memory pool and performs read and write operations on them.

Initially, I encountered issues with infinite page faults due to recursive access of metadata before it was mapped. This was resolved by allowing access to the metadata page during legitimacy checks. I also debugged issues related to incorrect address calculations and improper validation of memory ranges.

The final implementation correctly handles page faults, allocates frames on demand, and frees them when regions are released. The number of page faults corresponds to the number of pages accessed, confirming lazy allocation behavior. Memory contents were verified by writing and reading values, ensuring correctness of mappings.

```cpp
VMPool::VMPool(unsigned long _base_address, unsigned long _size,
               ContFramePool *_frame_pool, PageTable *_page_table) {
  assert(_size >= PageTable::PAGE_SIZE);
  base_address = _base_address;
  size = _size;
  frame_pool = _frame_pool;
  page_table = _page_table;

  allocated = (Region *)base_address;
  free_regions = (Region *)(base_address + MAX_REGIONS * sizeof(Region));

  n_alloc = 1;
  n_free = 1;

  page_table->register_pool(this);
  allocated[0].base = base_address;
  allocated[0].n_pages = 1;

  unsigned long total_pages = _size / PageTable::PAGE_SIZE;

  free_regions[0].base = base_address + PageTable::PAGE_SIZE;
  // Chose to round down here if requested memory is not a multiple of page size
  free_regions[0].n_pages = total_pages - 1;

  Console::puts("Constructed VMPool object.\n");
}
```

```cpp
unsigned long VMPool::allocate(unsigned long _size) {
  if (n_alloc >= MAX_REGIONS) {
    Console::puts("Out of memory regions.\n");
    return 0;
  }
  // Rounding up here
  unsigned long req_pages =
      (_size + PageTable::PAGE_SIZE - 1) / PageTable::PAGE_SIZE;

  if (req_pages == 0)
    return 0;

  for (unsigned long i = 0; i < n_free; i++) {
    if (free_regions[i].n_pages >= req_pages) {
      allocated[n_alloc].base = free_regions[i].base;
      allocated[n_alloc].n_pages = req_pages;
      n_alloc++;

      if (free_regions[i].n_pages == req_pages) {
        free_regions[i] = free_regions[n_free - 1];
        n_free--;
        Console::puts("Allocated region of memory.\n");
        return allocated[n_alloc - 1].base;
      }

      else {
        free_regions[i].base =
            free_regions[i].base + (req_pages * PageTable::PAGE_SIZE);
        free_regions[i].n_pages -= req_pages;

        Console::puts("Allocated region of memory.\n");
        return allocated[n_alloc - 1].base;
      }
    }
  }
  Console::puts("Allocated failed.\n");
  return 0;
}
```

```cpp
void VMPool::release(unsigned long _start_address) {
  if (n_free >= MAX_REGIONS) {
    Console::puts("Free memory list is already at max capacity.\n");
    return;
  }

  if (_start_address == base_address) {
    Console::puts("Error: Trying to free metadata page containing free list "
                  "and allocated list.\n");
    return;
  }

  for (unsigned long i = 0; i < n_alloc; i++) {
    if (allocated[i].base == _start_address) {
      unsigned long pages = allocated[i].n_pages;
      unsigned long base = allocated[i].base;

      for (unsigned long j = 0; j < pages; j++) {
        unsigned long page_no = (base / PageTable::PAGE_SIZE) + j;
        page_table->free_page(page_no);
      }

      free_regions[n_free] = allocated[i];
      allocated[i] = allocated[n_alloc - 1];
      n_alloc--;
      n_free++;
      Console::puts("Released region of memory.\n");
      return;
    }
  }

  Console::puts("No region found with the given start_address.\n");
}
```

```cpp
bool VMPool::is_legitimate(unsigned long _address) {
  if (_address >= base_address &&
      _address < base_address + PageTable::PAGE_SIZE) {
    Console::puts("The address is a part of the metadat region.\n");
    return true;
  }
  for (unsigned long i = 0; i < n_alloc; i++) {

    unsigned long start = allocated[i].base;
    unsigned long end = start + allocated[i].n_pages * PageTable::PAGE_SIZE;

    if (_address >= start && _address < end)
      return true;
  }
  Console::puts("The address is not a part of an allocated region.\n");
  return false;
}
```